Lesson 1 Lane Keeping

This lesson focuses on controlling the robot to move forward and maintain lane alignment through instructions.

1. Getting Ready

1) Before starting, ensure the map is laid out flat and free of wrinkles, with smooth roads and no obstacles. For specific map laying instructions, please refer to "Map Laying and Prop Installation" in the same directory as this section for guidance. (In this lesson, we are only experiencing the road driving-line patrol function, so there is no need to place props such as traffic lights and signboards.)

2) When experiencing this game, ensure it is conducted in a well-lit environment, but avoid direct light shining on the camera to prevent misrecognition.

3) It is essential to adjust the color threshold beforehand and set the color threshold of the yellow line to avoid misidentification during subsequent recognition. For specific color threshold adjustment, please refer to the "14. ROS+OpenCV Lesson" for reference.

4) It is recommended to position the robot in the middle of the road for easy identification!

2. Program Logic

Lane keeping can be divided into three parts: obtaining real-time images, image processing, and result comparison.

Firstly, real-time images are obtained by capturing images using the camera. Next, image processing includes color recognition, converting recognized images into different color spaces, erosion and dilation processing, and binarization processing.

The result comparison part involves processing the images to select the region of interest (ROI), outlining the processed images, and further comparing and calculating.

Finally, based on the comparison results, the direction of advancement is adjusted to keep the robot in the middle of the lane.

3. Operation Steps

Note: The input command should be case sensitive, and keywords can be complemented using Tab key.

1) Start the robot, and access the robot system desktop using VNC.

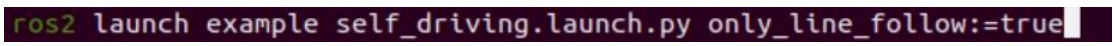
2) Click-on  to open the command-line terminal.

3) Execute the command to disable the app auto-start service.

```
~/.stop_sh
```

4) In the command line terminal, enter the command and press “Enter”.

```
ros2 launch examplt self_driving.launch.py onpy_line_follow:=true
```



5) Open a new command line terminal. Execute the command to open the interface for live camera feed.

```
rqt
```

6) If you need to terminate the game, press ‘Ctrl+C’. If the game cannot be terminated, please retry.

```
[odom_publisher-6] [INFO] [1723451164.349958435] [odom_publisher]: 0.930000 1.00
0000
[ekf_node-7] Failed to meet update rate! Took 0.164621822000000099seconds. Try
 decreasing the rate, limiting sensor output frequency, or limiting the number o
f sensors.
[odom_publisher-6] [INFO] [1723451165.054035094] [odom_publisher]: start
[apply_calib-11] [INFO] [1723451166.354463731] [imu_calib]: Gyro calibration com
plete! (bias = [0.024, -0.019, 0.004])
[ekf_node-7] Failed to meet update rate! Took 0.039219104999999997163seconds. Tr
y decreasing the rate, limiting sensor output frequency, or limiting the number
of sensors.
[servo_controller-8] [INFO] [1723451166.994503913] [servo_manager]: start
[ekf_node-7] Failed to meet update rate! Took 0.039147533999999997723seconds. Tr
y decreasing the rate, limiting sensor output frequency, or limiting the number
```

7) Once you've completed the game experience, you can initiate the app service by following the instructions or restarting the robot.

If the app service is not activated, the associated app functions will be disabled. (The app service will automatically start when the robot restarts).

To restart the app service, enter the following command and wait for the buzzer to emit a single beep, indicating that the service startup is complete.

```
sudo systemctl restart start_node.service
```

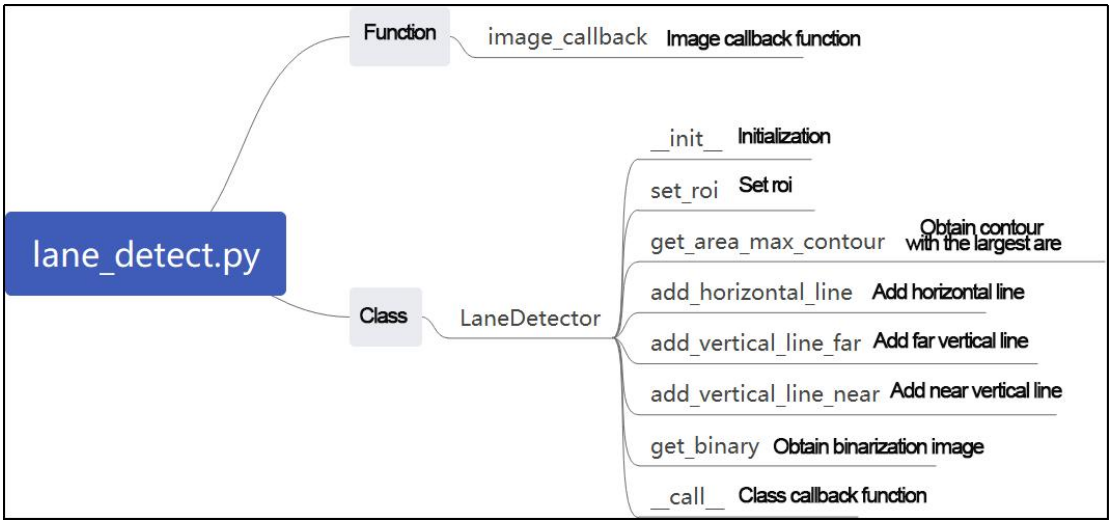
4. Program Outcome

After starting the game, place the robot on the road, and it will automatically detect the yellow line at the edge of the road. The robot will then adjust its position based on the detection results.

5. Program Analysis

The source code of this program is saved in

/home/ubuntu/ros2_ws/src/example/example/self_driving/lane_detect.py



Function:

image_callback：

```
173 def image_callback(ros_image):
174     cv_image = bridge.imgmsg_to_cv2(ros_image, "bgr8")
175     bgr_image = np.array(cv_image, dtype=np.uint8)
176     if image_queue.full():
177         # 如果队列已满，丢弃最旧的图像(if the queue is full, remove the oldest image)
178         image_queue.get()
179         # 将图像放入队列(put the image into the queue)
180     image_queue.put(bgr_image)
```

Image callback function: used to read camera node.

Class:

LaneDetector：

```
class LaneDetector(object):
    def __init__(self, color):
        # 车道线颜色(lane color)
        self.target_color = color
        # 车道线识别的区域(ROI for lane detection)
        if os.environ['DEPTH_CAMERA_TYPE'] == 'Dabai':
            self.rois = ((338, 360, 0, 320, 0.7), (292, 315, 0, 320, 0.2), (248, 270, 0, 320, 0.1))
        else:
            self.rois = ((450, 480, 0, 320, 0.7), (390, 480, 0, 320, 0.2), (330, 480, 0, 320, 0.1))
        self.weight_sum = 1.0

    def set_roi(self, roi):
        self.rois = roi
```

Init:

```
    def __init__(self, color):
        # 车道线颜色(lane color)
        self.target_color = color
        # 车道线识别的区域(ROI for lane detection)
        if os.environ['DEPTH_CAMERA_TYPE'] == 'Dabai':
            self.rois = ((338, 360, 0, 320, 0.7), (292, 315, 0, 320, 0.2), (248, 270, 0, 320, 0.1))
        else:
            self.rois = ((450, 480, 0, 320, 0.7), (390, 480, 0, 320, 0.2), (330, 480, 0, 320, 0.1))
        self.weight_sum = 1.0
```

Initialize the required parameters. Set ROI to lock recognition area.

set_roi:

```
    def set_roi(self, roi):
        self.rois = roi
```

Used to set recognized ROI.

get_area_max_contour：

```
    def get_area_max_contour(contours, threshold=100):
        '''
        获取最大面积对应的轮廓(obtain the contour corresponding to the maximum area)
        :param contours:
        :param threshold:
        :return:
        '''
        contour_area = zip(contours, tuple(map(lambda c: math.fabs(cv2.contourArea(c)), contours)))
        contour_area = tuple(filter(lambda c_a: c_a[1] > threshold, contour_area))
        if len(contour_area) > 0:
            max_c_a = max(contour_area, key=lambda c_a: c_a[1])
            return max_c_a
        return None
```

Get the contour list through OpenCV to obtain the contour with the maximum area.

add_horizontal_line：

```
    def add_horizontal_line(self, image):
        #   |-----  --->   |-----  ---> --
        h, w = image.shape[:2]
        roi_w_min = int(w/2)
        roi_w_max = w
        roi_h_min = 0
        roi_h_max = h
        roi = image[roi_h_min:roi_h_max, roi_w_min:roi_w_max]  # 截取右半边(crop the right half)
        flip_binary = cv2.flip(roi, 0)  # 上下翻转(flip upside down)
        max_y = cv2.minMaxLoc(flip_binary)[-1][1]  # 提取最上，最左数值为255的点坐标(extract the coordinates of the top-left point with a value of 255)

        return h - max_y
```

Set the recognition horizontal line based on the width and height of the image and ROI.

add_vertical_line_far：

```
    def add_vertical_line_near(self, image):
        # --|         |--       |
        #   |   --->  |    --->
        h, w = image.shape[:2]
        roi_w_min = 0
        roi_w_max = int(w/2)
        roi_h_min = int(h/2)
        roi_h_max = h
        roi = image[roi_h_min:roi_h_max, roi_w_min:roi_w_max]
        flip_binary = cv2.flip(roi, -1)  # 图像左右上下翻转(flip the image horizontally and vertically)
        #cv2.imshow('1', flip_binary)
        (x_0, y_0) = cv2.minMaxLoc(flip_binary)[-1]  # 提取最上，最左数值为255的点坐标(extract the coordinates of the top-left point with a value of 255)
        down_p = (roi_w_max - x_0, roi_h_max - y_0)

        (x_1, y_1) = cv2.minMaxLoc(roi)[-1]
        y_center = int((roi_h_max - roi_h_min - y_1 + y_0)/2)
        roi = flip_binary[y_center:, :]
        (x, y) = cv2.minMaxLoc(roi)[-1]
        up_p = (roi_w_max - x, roi_h_max - (y + y_center))

        up_point = (0, 0)
        down_point = (0, 0)
        if up_p[1] - down_p[1] != 0 and up_p[0] - down_p[0] != 0:
            up_point = (int(-down_p[1]/((up_p[1] - down_p[1])/(up_p[0] - down_p[0])) + down_p[0]), 0)
            down_point = down_p

        return up_point, down_point, y_center
```

Set the recognition vertical line based on the ROI and the distance from the robot to the farthest part of the image.

get_binary

```
    def get_binary(self, image):
        # 通过lab空间识别颜色(recognize color through LAB space)
        img_lab = cv2.cvtColor(image, cv2.COLOR_RGB2LAB)  # rgb转lab(convert RGB to LAB)
        img_blur = cv2.GaussianBlur(img_lab, (3, 3), 3)  # 高斯模糊去噪(Gaussian blur denoising)
        mask = cv2.inRange(img_blur, tuple(lab_data['lab']['Stereo'][self.target_color]['min']), tuple(lab_data['lab']['Stereo'][self.target_color]['max']))  # 二值化(binarization)
        eroded = cv2.erode(mask, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 腐蚀(erode)
        dilated = cv2.dilate(eroded, cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3)))  # 膨胀(dilate)

        return dilated
```

Perform color recognition based on color space, and process a binarized image.

add_vertical_line_near：

```
    def add_vertical_line_near(self, image):
        # --|         |--       |
        #   |   --->  |    --->
        h, w = image.shape[:2]
        roi_w_min = 0
        roi_w_max = int(w/2)
        roi_h_min = int(h/2)
        roi_h_max = h
        roi = image[roi_h_min:roi_h_max, roi_w_min:roi_w_max]
        flip_binary = cv2.flip(roi, -1)  # 图像左右上下翻转(flip the image horizontally and vertically)
        #cv2.imshow('1', flip_binary)
        (x_0, y_0) = cv2.minMaxLoc(flip_binary)[-1]  # 提取最上，最左数值为255的点坐标(extract the coordinates of the top-left point with a value of 255)
        down_p = (roi_w_max - x_0, roi_h_max - y_0)

        (x_1, y_1) = cv2.minMaxLoc(roi)[-1]
        y_center = int((roi_h_max - roi_h_min - y_1 + y_0)/2)
        roi = flip_binary[y_center:, :]
        (x, y) = cv2.minMaxLoc(roi)[-1]
        up_p = (roi_w_max - x, roi_h_max - (y + y_center))

        up_point = (0, 0)
        down_point = (0, 0)
        if up_p[1] - down_p[1] != 0 and up_p[0] - down_p[0] != 0:
            up_point = (int(-down_p[1]/((up_p[1] - down_p[1])/(up_p[0] - down_p[0])) + down_p[0]), 0)
            down_point = down_p

        return up_point, down_point, y_center
```

Set a vertical recognition line that is closer to the robot based on the ROI and image width and height.

__call__：

```python
    def __call__(self, image, result_image):
        # 按比重提取线中心(extract the center point based on the proportion)
        centroid_sum = 0
        h, w = image.shape[:2]
        max_center_x = -1
        center_x = []
        for roi in self.rois:
            blob = image[roi[0]:roi[1], roi[2]:roi[3]]  # 截取roi(crop ROI)
            contours = cv2.findContours(blob, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_TC89_L1)[-2]  # 找轮廓(find contours)
            max_contour_area = self.get_area_max_contour(contours, 30)  # 获取最大面积对应轮廓(obtain the contour with the largest area)
            if max_contour_area is not None:
                rect = cv2.minAreaRect(max_contour_area[0])  # 最小外接矩形(the minimum bounding rectangle)
                box = np.intp(cv2.boxPoints(rect))  # 四个角(four corners)
                for j in range(4):
                    box[j, 1] = box[j, 1] + roi[0]
                cv2.drawContours(result_image, [box], -1, (255, 255, 0), 2)  # 画出四个点组成的矩形(draw the rectangle composed of the four points)

                # 获取矩形对角点(obtain the diagonal points of the rectangle)
                pt1_x, pt1_y = box[0, 0], box[0, 1]
                pt3_x, pt3_y = box[2, 0], box[2, 1]
                # 线的中心点(the center point of the line)
                line_center_x, line_center_y = (pt1_x + pt3_x) / 2, (pt1_y + pt3_y) / 2

                cv2.circle(result_image, (int(line_center_x), int(line_center_y)), 5, (0, 0, 255), -1)  # 画出中心点(draw the center point)
                center_x.append(line_center_x)
            else:
                center_x.append(-1)
        for i in range(len(center_x)):
            if center_x[i] != -1:
                if center_x[i] > max_center_x:
                    max_center_x = center_x[i]
                centroid_sum += center_x[i] * self.rois[i][-1]
        if centroid_sum == 0:
            return result_image, None, max_center_x
        center_pos = centroid_sum / self.weight_sum  # 按比重计算中心点(calculate the center point based on the proportion)
        angle = math.degrees(-math.atan((center_pos - (w / 2.0)) / (h / 2.0)))

        return result_image, angle, max_center_x
```

The callback function of the entire class: In this function, color recognition is performed. The detected yellow line is drawn using OpenCV. The output includes the image, angle, and pixel coordinates X of the recognized contours in each ROI.